CS224W Homework 2

Due: October 30, 2025

1 GNNs as MLP of eigenvectors [20 points]

Coverage: This problem is covered in Lecture 7.

In this problem, we analyze message-passing GNN layers from a spectral perspective. We will (i) rewrite per-node updates in a batch matrix form, (ii) specialize to a single-layer MLP, and (iii) express the layer outputs in the eigenvector basis of the (symmetric) adjacency matrix.

1.1 Batch Node Update [2 points]

Consider the update for Graph Isomorphism Network (GIN):

$$\mathbf{x}_v^{(l+1)} = \texttt{MLP}\left((1+\epsilon)\,\mathbf{x}_v^{(l)} + \sum_{u\in\mathcal{N}(v)} \mathbf{x}_u^{(l)}\right), \tag{1}$$

where $\mathbf{x}_v^{(l)} \in \mathbb{R}^{d_l}$ is the embedding of node v at layer l. Let $\mathbf{X}^{(l)} \in \mathbb{R}^{N\times d_l}$ be a matrix containing the embeddings of all the nodes in the graph, i.e., $\mathbf{X}^{(l)}[v,:] = \mathbf{x}_v^{(l)\top}$. Also, let $\mathbf{A} \in \{0,1\}^{N\times N}$ represent the adjacency matrix of the graph. Write down the update of $\mathbf{X}^{(l+1)}$ as a function of $\mathbf{X}^{(l)}$ and $\mathbf{A}$.

★ Solution ★

$$\mathbf{X}^{(l+1)} = \texttt{MLP}\left((\mathbf{A} + (1+\epsilon)\mathbf{I})\,\mathbf{X}^{(l)}\right)$$

1.2 Single Layer MLP [2 points]

Assume that $\texttt{MLP}()$ represents a single layer MLP with no bias term followed by a pointwise nonlinearity σ. Write down the update of $\mathbf{X}^{(l+1)}$ as a function of $\mathbf{X}^{(l)}$ and $\mathbf{A}$, and the trainable parameters $\mathbf{W}^{(l)}$ of layer l.

★ Solution ★

$$\mathbf{X}^{(l+1)} = \sigma\left((\mathbf{A} + (1+\epsilon)\mathbf{I})\,\mathbf{X}^{(l)}\mathbf{W}^{(l)}\right)$$

1.3 Eigenvector Extension [4 points]

Let $\{\lambda_n, \mathbf{v}_n\}_{n=1}^N$ represent the eigenvalues and eigenvectors of the symmetric adjacency $\mathbf{A}$. Then we can write $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$, where $\mathbf{V} \in \mathbb{R}^{N\times N}$ is the matrix of eigenvectors with $\mathbf{V}[:, n] = \mathbf{v}_n$ and $\mathbf{\Lambda} \in \mathbb{R}^{N\times N}$ is the diagonal matrix of eigenvalues with $\mathbf{\Lambda}[n, n] = \lambda_n$. Show that

$$\mathbf{X}^{(l+1)} = \sigma\left(\mathbf{V}\hat{\mathbf{W}}^{(l)}\right)$$

where $\hat{\mathbf{W}}^{(l)}$ has entries

$$\hat{\mathbf{W}}^{(l)}[n, j] \;=\; \left(\lambda_n + 1 + \epsilon\right) \sum_{i=1}^{d_l} \mathbf{W}^{(l)}[i, j]\, \left\langle \mathbf{v}_n,\, \mathbf{X}^{(l)}[:, i]\right\rangle,$$

and $\langle\cdot\rangle$ denotes the dot product.

Hint: Use the fact that the eigenvectors of a symmetric matrix form an orthonormal basis. In other words, the matrix $\mathbf{V}$ of eigenvectors is orthogonal, i.e., $\mathbf{V}^\top\mathbf{V} = \mathbf{V}\mathbf{V}^\top = \mathbf{I}$. Next, show that each feature across all nodes, $\mathbf{X}^{(l+1)}[:, i]$, can be expressed as a linear combination of eigenvectors, followed by a pointwise nonlinearity.

★ Solution ★

$$\begin{aligned}\mathbf{X}^{(l+1)} &= \sigma\left(\left(\mathbf{A} + (1+\epsilon)\mathbf{I}\right)\mathbf{X}^{(l)}\mathbf{W}^{(l)}\right)\\ &= \sigma\left(\mathbf{V}\left(\mathbf{\Lambda} + (1+\epsilon)\mathbf{I}\right)\mathbf{V}^\top\mathbf{X}^{(l)}\mathbf{W}^{(l)}\right)\end{aligned}$$

set

$$\hat{\mathbf{W}}^{(l)} = \left(\mathbf{\Lambda} + (1+\epsilon)\mathbf{I}\right)\mathbf{V}^\top\mathbf{X}^{(l)}\mathbf{W}^{(l)}$$

, so

$$\begin{aligned}\hat{\mathbf{W}}^{(l)}[n, j] &= \left(\mathbf{\Lambda} + (1+\epsilon)\mathbf{I}\right)[n, n]\left(\mathbf{V}^\top\mathbf{X}^{(l)}\mathbf{W}^{(l)}\right)[n, j]\\ &= \left(\lambda_n + 1 + \epsilon\right)\sum_{i=1}^{d_l}\left(\mathbf{V}^\top\mathbf{X}^{(l)}\right)[n, i]\mathbf{W}^{(l)}[i, j]\\ &= \left(\lambda_n + 1 + \epsilon\right)\sum_{i=1}^{d_l}\mathbf{W}^{(l)}[i, j]\left\langle\mathbf{v}_n,\, \mathbf{X}^{(l)}[:, i]\right\rangle\end{aligned}$$

$$\mathbf{X}^{(l+1)}[:, j] = \sigma\left(\mathbf{V}\hat{\mathbf{W}}^{(l)}\right)[:, j] = \sigma\left(\sum_{n=1}^{N}\hat{\mathbf{W}}^{(l)}[n, j]\mathbf{v}_n\right)$$

1.4 GraphSAGE [4 points]

Perform the same analysis for the GraphSAGE update when the aggregation function is sum pooling. *i.e.*, derive (i) $\mathbf{X}^{(l+1)}$ and (ii) $\hat{\mathbf{W}}^{(l)}[n, j]$. Recall that the GraphSAGE update function is

$$\begin{aligned}\mathbf{x}_v^{(l+1)} &= \sigma\left(\mathbf{W}^{(l)} \cdot \text{CONCAT}\left(\mathbf{x}_v^{(l)}, \mathbf{x}_{N(v)}^{(l)}\right)\right) \\ &= \sigma\left(\mathbf{W}_1^{(l)}\mathbf{x}_v^{(l)} + \mathbf{W}_2^{(l)}\text{AGG}\left(\mathbf{x}_u^{(l)}, \forall u \in N(v)\right)\right)\end{aligned}$$

★ Solution ★

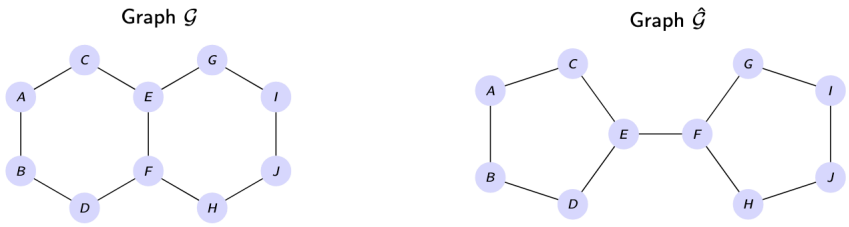
$$\begin{aligned}\mathbf{X}^{(l+1)} &= \sigma\left(\mathbf{X}^{(l)}\mathbf{W}_1^{(l)} + \mathbf{A}\mathbf{X}^{(l)}\mathbf{W}_2^{(l)}\right) \\ &= \sigma\left(\mathbf{V}\left(\mathbf{V}^\top\mathbf{X}^{(l)}\mathbf{W}_1^{(l)} + \mathbf{\Lambda}\mathbf{V}^\top\mathbf{X}^{(l)}\mathbf{W}_2^{(l)}\right)\right)\end{aligned}$$

set

$$\hat{\mathbf{W}}^{(l)} = \mathbf{V}^\top\mathbf{X}^{(l)}\mathbf{W}_1^{(l)} + \mathbf{\Lambda}\mathbf{V}^\top\mathbf{X}^{(l)}\mathbf{W}_2^{(l)}$$

$$\begin{aligned}\hat{\mathbf{W}}^{(l)}[n, j] &= \left(\mathbf{V}^\top\mathbf{X}^{(l)}\mathbf{W}_1^{(l)}\right)[n, j] + \mathbf{\Lambda}[n, n]\left(\mathbf{V}^\top\mathbf{X}^{(l)}\mathbf{W}_2^{(l)}\right)[n, j] \\ &= \sum_{i=1}^{d_l}\left(\mathbf{V}^\top\mathbf{X}^{(l)}\right)[n, i]\mathbf{W}_1^{(l)}[i, j] + \lambda_n\sum_{i=1}^{d_l}\left(\mathbf{V}^\top\mathbf{X}^{(l)}\right)[n, i]\mathbf{W}_2^{(l)}[i, j] \\ &= \sum_{i=1}^{d_l}\mathbf{W}_1^{(l)}[i, j]\langle\mathbf{v}_n,\ \mathbf{X}^{(l)}[:, i]\rangle + \lambda_n\sum_{i=1}^{d_l}\mathbf{W}_2^{(l)}[i, j]\langle\mathbf{v}_n,\ \mathbf{X}^{(l)}[:, i]\rangle\end{aligned}$$

1.5 Eigendecomposition Analysis [8 points]



For graphs $\mathcal{G}$ and $\hat{\mathcal{G}}$ instantiate the graph adjacencies in Numpy, PyTorch, or PyG, and compute their eigenvalue decompositions. What do you observe? [2 points]

★ Solution ★ Their eigenvalues are almost same.

Consider a GIN where all nodes start with the same initial color, i.e., $\mathbf{x}_v^{(0)} = 1$ for all nodes $v \in \mathcal{V}$. This setup is equivalent to having $\mathbf{X}^{(0)} = \mathbf{1}$, where $\mathbf{1}$ denotes the all-one vector. This is the initialization of the WL test. Using the equations in 1.3, derive the expression for $\mathbf{X}^{(1)}$. [2 points]

★ **Solution** ★ Given the conditions, we have $d_0 = 1$

$$\begin{aligned}\hat{\mathbf{W}}^{(0)}[n, j] &= (\lambda_n + 1 + \epsilon) \sum_{i=1}^{d_0} \mathbf{W}^{(0)}[i, j] \langle \mathbf{v}_n,\ \mathbf{X}^{(0)}[:, i] \rangle \\ &= (\lambda_n + 1 + \epsilon) \mathbf{W}^{(0)}[1, j] \langle \mathbf{v}_n,\ \mathbf{1} \rangle\end{aligned}$$

$$\begin{aligned}\mathbf{X}^{(1)}[:, j] &= \sigma\left(\sum_{n=1}^{N} \hat{\mathbf{W}}^{(0)}[n, j] \mathbf{v}_n \right) \\ &= \sigma\left(\mathbf{W}^{(0)}[1, j] \sum_{n=1}^{N} (\lambda_n + 1 + \epsilon) \langle \mathbf{v}_n,\ \mathbf{1} \rangle \mathbf{v}_n \right)\end{aligned}$$

Observe from your previous result that each column $\mathbf{X}^{(1)}[:, j]$ is a linear combination of eigenvectors, followed by a pointwise nonlinearity. What is the weight associated with each eigenvector? What factors determine this weight? [2 points]

★ **Solution** ★ $\mathbf{W}^{(0)}[1, j](\lambda_n + 1 + \epsilon) \langle \mathbf{v}_n,\ \mathbf{1} \rangle$ is the weight associated with each eigenvector. Graph structure and the parameters in the network determine this weight.

Compute the dot product $\langle \mathbf{v}_n, \mathbf{X}^{(0)} \rangle$, for each eigenvector across both graphs. In other words, compute the dot products $c_n = \langle \mathbf{v}_n, \mathbf{1}_N \rangle$ for $\mathcal{G}$ and $\hat{c}_n = \langle \hat{\mathbf{v}}_n, \mathbf{1}_N \rangle$ for $\hat{\mathcal{G}}$. What do you observe? [1 point]

★ **Solution** ★ They are almost same.

What does the previous result suggest about $\mathbf{X}^{(1)}$ for the graphs $\mathcal{G}$ and $\hat{\mathcal{G}}$? [1 point]

★ **Solution** ★ $\mathbf{X}^{(1)}$ for the graphs $\mathcal{G}$ and $\hat{\mathcal{G}}$ would be same.

2 Node Embeddings with TransE [21 points]

Coverage: This problem is covered in Lecture 10.

While many real world systems are effectively modeled as graphs, graphs can be a cumbersome format for certain downstream applications, such as machine learning models. It is often useful to represent each node of a graph as a vector in

a continuous low dimensional space. The goal is to preserve information about the structure of the graph in the vectors assigned to each node. For instance, the spectral embedding preserved structure in the sense that nodes connected by an edge were usually close together in the (one-dimensional) embedding x.

Multi-relational graphs are graphs with multiple types of edges. They are incredibly useful for representing structured information, as in knowledge graphs. There may be one node representing "Washington, DC" and another representing "United States", and an edge between them with the type "Is capital of". In order to create an embedding for this type of graph, we need to capture information about not just which edges exist, but what the types of those edges are. In this problem, we will explore a particular algorithm designed to learn node embeddings for multi-relational graphs.

The algorithm we will look at is TransE.[1] We will first introduce some notation used in the paper describing this algorithm. We'll let a multi-relational graph $G = (E, S, L)$ consist of the set of *entities* E (i.e., nodes), a set of edges S, and a set of possible relationships L. The set S consists of triples (h, l, t), where $h \in E$ is the *head* or source-node, $l \in L$ is the relationship, and $t \in E$ is the *tail* or destination-node. As a node embedding, TransE tries to learn embeddings of each entity $e \in E$ into $\mathbb{R}^k$ (k-dimensional vectors), which we will notate by $\mathbf{e}$. The main innovation of TransE is that each relationship ℓ is also embedded as a vector $\ell \in \mathbb{R}^k$, such that the difference between the embeddings of entities linked via the relationship ℓ is approximately ℓ. That is, if $(h, \ell, t) \in S$, TransE tries to ensure that $\mathbf{h} + \boldsymbol{\ell} \approx \mathbf{t}$. Simultaneously, TransE tries to make sure that $\mathbf{h} + \boldsymbol{\ell} \not\approx \mathbf{t}$ if the edge (h, ℓ, t) does not exist.

Note on notation: we will use unbolded letters e, ℓ, etc. to denote the entities and relationships in the graph, and bold letters $\mathbf{e}, \boldsymbol{\ell}$, etc., to denote their corresponding embeddings. TransE accomplishes this by minimizing the following loss:

$$\mathcal{L} = \sum_{(h,\ell,t)\in S} \left(\sum_{(h',\ell,t')\in S'_{(h,\ell,t)}} [\gamma + d(\mathbf{h} + \boldsymbol{\ell}, \mathbf{t}) - d(\mathbf{h}' + \boldsymbol{\ell}, \mathbf{t}')]_+ \right) \tag{2}$$

Here (h', ℓ, t') are "corrupted" triplets, chosen from the set $S'_{(h,\ell,t)}$ of corruptions of (h, ℓ, t), which are all triples where either h or t (but not both) is replaced by a random entity, and ℓ remains the same as the one in the original triplets.

$$S'_{(h,\ell,t)} = \{(h', \ell, t) \mid h' \in E\} \cup \{(h, \ell, t') \mid t' \in E\}$$

Additionally, $\gamma > 0$ is a fixed scalar called the *margin*, the function $d(\cdot, \cdot)$ is the Euclidean distance, and $[\cdot]_+$ is the positive part function (defined as $\max(0, \cdot)$).

[1] See the 2013 NeurIPS paper by Bordes et al: https://papers.nips.cc/paper/5071-translating-embeddings-formodeling-multi-relational-data.pdf

Finally, TransE restricts **all the entity embeddings to have length** 1 : $\|\mathbf{e}\|_2 = 1$ **for every** $e \in E$**.**
For reference, here is the TransE algorithm, as described in the original paper on page 3:

Algorithm 1 Learning TransE

input Training set $S = \{(h, \ell, t)\}$, entities and rel. sets E and L, margin γ, embeddings dim. k.

initialize $\boldsymbol{\ell} \leftarrow$ uniform$(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each $\ell \in L$
  $\boldsymbol{\ell} \leftarrow \boldsymbol{\ell} / \|\boldsymbol{\ell}\|$ for each $\ell \in L$
  $\mathbf{e} \leftarrow$ uniform$(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each entity $e \in E$
loop
  $\mathbf{e} \leftarrow \mathbf{e} / \|\mathbf{e}\|$ for each entity $e \in E$
  $S_{batch} \leftarrow$ sample(S, b) // sample a minibatch of size b
  $T_{batch} \leftarrow \emptyset$ // initialize the set of pairs of triplets
  for $(h, \ell, t) \in S_{batch}$ **do**
    $(h', \ell, t') \leftarrow$ sample$(S'_{(h,\ell,t)})$ // sample a corrupted triplet
    $T_{batch} \leftarrow T_{batch} \cup \{((h, \ell, t), (h', \ell, t'))\}$
  end for
  Update embeddings w.r.t. $\sum_{((h,\ell,t),(h',\ell,t')) \in T_{batch}} \nabla \left[\gamma + d(\boldsymbol{h} + \boldsymbol{\ell}, \boldsymbol{t}) - d(\boldsymbol{h'} + \boldsymbol{\ell}, \boldsymbol{t'})\right]_+$
end loop

2.1 Simplified Objective [3 points]

Say we were intent on using a simpler loss function. Our objective function (2) includes a term maximizing the distance between $\mathbf{h}' + \boldsymbol{\ell}$ and $\mathbf{t}'$. If we instead simplified the objective, and just tried to minimize

$$\mathcal{L}_{\text{simple}} = \sum_{(h,\ell,t) \in S} d(\mathbf{h} + \boldsymbol{\ell}, \mathbf{t}), \tag{3}$$

we would obtain a useless embedding. Give an example of a simple graph and corresponding embeddings which will minimize the new objective function (3) all the way to zero, but still give a completely useless embedding.
Hint: Your graph should be non-trivial, i.e., it should include at least two nodes and at least one edge. Assume the embeddings are in 2 dimensions, i.e., $k = 2$. What happens if $\boldsymbol{\ell} = \mathbf{0}$?

★ Solution ★
Assume a graph with 2 node and 1 edge, where $\mathbf{h}_a = \mathbf{h}_b = [1, 0]^\top$, $\boldsymbol{\ell} = \mathbf{0}$.

2.2 Utility of γ [5 points]

We are interested in understanding what the margin term γ accomplishes. If we removed the margin term γ from our loss, and instead optimized

$$\mathcal{L}_{\text{no margin}} = \sum_{(h,\ell,t) \in S} \sum_{(h',\ell,t') \in S'_{(h,\ell,t)}} \left[d(\mathbf{h} + \boldsymbol{\ell}, \mathbf{t}) - d\left(\mathbf{h}' + \boldsymbol{\ell}, \mathbf{t}'\right)\right]_+, \tag{4}$$

it turns out that we would again obtain a useless embedding. Give an example of a simple graph and corresponding embeddings which will minimize the new objective function (4) all the way to zero, but still give a completely useless embedding. By useless, we mean that in your example, you cannot tell just from the embeddings whether two nodes are linked by a particular relation (Note: your graph should be non-trivial, i.e., it should include at least two nodes and at least one edge. Assume the embeddings are in 2 dimensions, i.e., $k = 2$.)

★ Solution ★

2.3 Normalizing the embeddings [5 points]

Recall that TransE normalizes every entity embedding to have unit length (see line 5 of the algorithm). The quality of our embeddings would be much worse if we did not have this step. To understand why, imagine running the algorithm with line 5 omitted. What could the algorithm do to trivially minimize the loss in this case? What would the embeddings it generates look like?

★ Solution ★

When the embeddings satisfy $d(\mathbf{h}+\boldsymbol{\ell}, \mathbf{t}) - d(\mathbf{h}'+\boldsymbol{\ell}, \mathbf{t}') < 0$, the algorithm could infinitely scale up them to make the objective function to 0 and have no sense for the actual embedding structure.

2.4 Expressiveness of TransE embeddings [8 points]

Give an example of a simple graph for which no perfect embedding exists, i.e., no embedding perfectly satisfies $\mathbf{u} + \boldsymbol{\ell} = \mathbf{v}$ for all $(u, \ell, v) \in S$ and $\mathbf{u} + \boldsymbol{\ell} \neq \mathbf{v}$ for $(u, \ell, v) \notin S$, for any choice of entity embeddings ($\mathbf{e}$ for $e \in E$) and relationship embeddings ($\boldsymbol{\ell}$ for $\ell \in L$). Explain why this graph has no perfect embedding in this system, and what that means about the expressiveness of TransE embeddings. As before, assume the embeddings are in 2 dimensions ($k = 2$).

Hint: Note that the condition for this question is slightly different from that for Question 2.1 and what we ask you to answer is different as well.

★ Solution ★

Consider a graph with two entities A, B and one relation ℓ, where $S = \{(A, \ell, A), (A, \ell, B)\}$, but $(B, \ell, A) \notin S$.

A perfect TransE embedding must satisfy $\mathbf{a} + \boldsymbol{\ell} = \mathbf{a}$ and $\mathbf{a} + \boldsymbol{\ell} = \mathbf{b}$. The first equation implies $\boldsymbol{\ell} = \mathbf{0}$, and the second implies $\mathbf{a} = \mathbf{b}$. Consequently, $\mathbf{b} + \boldsymbol{\ell} = \mathbf{a}$, incorrectly satisfying the triple $(B, \ell, A) \notin S$.

Therefore, no perfect TransE embedding exists for this graph. This demonstrates that TransE has limited expressiveness and cannot represent all possible relational structures.

3 Expressive Power of Knowledge Graph Embeddings [10 points]

Coverage: This problem is covered in Lecture 10.

TransE is a common method for learning representations of entities and relations in a knowledge graph. Given a triplet (h, ℓ, t), where entities embedded as h and t are related by a relation embedded as ℓ, TransE trains entity and relation embeddings to make $h + \ell$ close to t. There are some common patterns that relations form:

- Symmetry: A is married to B, and B is married to A.
- Inverse: A is teacher of B, and B is student of A. Note that teacher and student are 2 different relations and have their own embeddings.
- Composition: A is son of B; C is sister of B, then C is aunt of A. Again note that son, sister, and aunt are 3 different relations and have their own embeddings.

3.1 TransE Modeling [3 points]

For each of the above relational patterns, can TransE model it perfectly, such that $h + \ell = t$ for all relations? Explain why or why not. Note that here $\mathbf{0}$ embeddings for relation are undesirable since that means two entities related by that relation are identical and not distinguishable.

★ Solution ★

1. TransE can't model relation of symmetry, since

$$\mathbf{h} + \boldsymbol{\ell} = \mathbf{t}$$

$$\mathbf{t} + \boldsymbol{\ell} = \mathbf{h} + 2\boldsymbol{\ell} \neq \mathbf{h}$$

2. TransE can model relation of inverse, for example set $\boldsymbol{\ell}_1 = -\boldsymbol{\ell}_2$, when

$$\mathbf{h} + \boldsymbol{\ell}_1 = \mathbf{t}$$

$$\mathbf{t} + \boldsymbol{\ell}_2 = \mathbf{h}$$

3. TransE can model relation of composition, set $\boldsymbol{\ell}_3 = \boldsymbol{\ell}_1 + \boldsymbol{\ell}_2$, when

$$\mathbf{a} + \boldsymbol{\ell}_1 = \mathbf{b}$$

$$\mathbf{b} + \boldsymbol{\ell}_2 = \mathbf{c}$$

, we have

$$\mathbf{a} + \boldsymbol{\ell}_3 = \mathbf{c}$$

3.2 RotatE Modeling [3 points]

Consider a new model, RotatE. Instead of training embeddings such that $h+\ell \approx t$, we train embeddings such that $h \circ \ell \approx t$. Here $\circ$ means rotation. You can think of h as a vector of dimension $2d$, representing d 2D points. ℓ is a d-dimensional vector specifying rotation angles. When applying $\circ$, For all $i \in 0 \ldots d-1, (h_{2i}, h_{2i+1})$ is rotated clockwise by l_i. Similar to TransE, the entity embeddings are also normalized to L2 norm 1. Can RotatE model the above 3 relation patterns perfectly such that $h \circ \ell = t$ for all relations? Why or why not?

★ Solution ★

1. RotatE can model relation of symmetry, set $l_i = 2k\pi, \ \ k \in \mathbb{Z}$, so

$$\mathbf{h} \circ \boldsymbol{\ell} = \mathbf{t}$$

$$\mathbf{t} \circ \boldsymbol{\ell} = \mathbf{h}$$

2. RotatE can model relation of inverse, set $l_i^{(2)} = -l_i^{(1)} + 2k\pi, \ \ k \in \mathbb{Z}$, so

$$\mathbf{h} \circ \boldsymbol{\ell}_1 = \mathbf{t}$$

$$\mathbf{t} \circ \boldsymbol{\ell}_2 = \mathbf{h}$$

3. RotatE can model relation of inverse, set $l_i^{(3)} = l_i^{(1)} + l_i^{(2)} + 2k\pi, \ \ k \in \mathbb{Z}$, when

$$\mathbf{a} \circ \boldsymbol{\ell}_1 = \mathbf{b}$$

$$\mathbf{b} \circ \boldsymbol{\ell}_2 = \mathbf{c}$$

, so

$$\mathbf{a} \circ \boldsymbol{\ell}_3 = \mathbf{c}$$

3.3 Failure Cases [4 points]

Give an example of a graph that RotatE cannot model. Can TransE model this graph? Assume that relation embeddings cannot be $\mathbf{0}$ in either model.

★ Solution ★

4 Honor Code [0 points]

(X) I have read and understood Stanford Honor Code before I submitted my work.

Collaboration: Write down the names & SUNetIDs of students you collaborated with on Homework 2 (None if you didn't).

Note: Read our website on our policy about collaboration!